Agentic HiL: A Multi-Agent LLM Framework for Autonomous Generation of FPGA-based Real-Time Power Electronics Simulators

Qingcheng Sui[1], Yang Li[1], Jiaze Kong[1], Yu Zuo[1], Wilmar Martinez[1]
[1]Department of Electrical Engineering, KU Leuven - EnergyVille
Thor Park 8310, Genk, Belgium
{qingcheng.sui, yang.li1, jiaze.kong, yu.zuo, wilmar.martinez}@kuleuven.be

***Abstract*—Developing FPGA-based real-time Hardware-in-the-Loop (HiL) simulators for power electronics requires expertise spanning circuit modeling, numerical simulation, and hardware description languages, creating a significant barrier for engineers without cross-domain expertise. Although Large Language Models (LLMs) have demonstrated potential in automating digital hardware design, existing LLM-driven frameworks lack mechanisms for handling continuous-time dynamics, numerical stability, and hard real-time constraints. This paper introduces Agentic HiL, a multi-agent LLM framework that assists engineers in generating FPGA-based real-time power electronic simulators directly from circuit topologies. The framework integrates: (i) physics-aware modeling with automatic discretization and fixed-point optimization; (ii) waveform-driven hardware verification using RMSE-based comparison against floating-point references; and (iii) latency-aware synthesis optimization satisfying sub-microsecond simulation step requirements. The framework is validated on a diode-rectified Buck converter ($f_{sw} = 20$ kHz, 50 ns step) and a Boost converter implemented on a Zynq-7020 FPGA. The generated RTL achieves a pipeline latency of 22.2 ns and a critical-path delay of 5.6 ns, while maintaining a steady-state error of 4.87 mV ($< 0.04\%$) with only 156 LUTs (0.29%) and 3 DSP48E1 slices. Agentic HiL facilitates the translation of physical system descriptions into deterministic FPGA implementations, reducing the expertise barrier for real-time power electronic simulator development.**

***Index Terms*—FPGA, hardware-in-the-loop, large language models, multi-agent systems, power electronics, real-time simulation**

I. Introduction

The growing applications of wide-bandgap semiconductors push modern power electronic converters into hundreds of kilohertz ranges, demanding Hardware-in-the-Loop (HiL) simulation step sizes (Δt) at the sub-microsecond level [1]. To meet these severe deterministic latency constraints, Field-Programmable Gate Arrays (FPGAs) have become a dominant computing platform [2]–[5] for real-time power electronics simulators. Device-level FPGA modeling [6] and high-resolution IGBT behavioral models [7] have demonstrated the capability to resolve fast switching transients at the nanosecond scale. Meanwhile, open-source rapid control prototyping (RCP) platforms [8] and FPGA-based controllers [9], [10] demonstrate that modern control algorithms can be deployed directly on the same programmable fabric, motivating a co-located architecture where plant simulator and controller share a single FPGA device. However, developing such FPGA-based real-time simulators requires translating continuous-time physics equations into highly parallel, fixed-point hardware description languages (HDL) like Verilog. This introduces a steep programming barrier for power electronics engineers.

Simultaneously, Large Language Models (LLMs) are reshaping Electronic Design Automation (EDA). Multi-agent frameworks such as AutoGen [11] and ReAct-prompted agents [12] have been adopted for hardware design, with NVIDIA's VerilogCoder [13] utilizing AST-based code analysis and waveform-guided debugging to autonomously generate synthesizable digital logic. However, these AI agents are not specifically designed for physics-aware numerical modeling. They primarily target functional correctness of digital logic, e.g., finite state machines (FSMs), and cannot translate continuous dynamic equations into numerically stable, fixed-point hardware under strict real-time boundaries.

Existing industrial workflows such as MATLAB/Simulink HDL Coder and the Simscape HDL Workflow Advisor significantly simplify FPGA deployment by automatically translating block-diagram models into synthesizable HDL. However, these approaches assume that the physical model, numerical discretization strategy, and fixed-point representation have already been manually established by domain experts—precisely the steps that our Physics-Aware Modeling Agent automates.

To bridge this gap, we propose the Agentic HiL Coder, a multi-agent framework tailored for FPGA-based power electronic simulator development (Fig. 1). Inspired by VerilogCoder, we extend the paradigm into the physical domain with three distinct contributions:

1) **Physics-Aware Modeling Agent:** Automatically transforms circuit topologies into discrete-time numerical formulations, including state-space and ADC-MNA representations, performs discretization, and determines suitable fixed-point Q-format representations.
2) **Waveform-Driven Hardware Agent:** Extends conventional HDL debugging approaches with physics-aware waveform verification by comparing hardware outputs against floating-point golden references using RMSE-based metrics, enabling identification of numerical and quantization-related errors.

3) **Automated Synthesis Loop:** Integrates synthesis-aware optimization by monitoring critical-path timing and ensuring that hardware computation latency (t_{calc}) satisfies the hard real-time integration-step constraint ($t_{calc} \ll \Delta t$).

II. Proposed Agentic HiL Framework

The architecture of the proposed Agentic HiL Coder consists of interconnected specialized agents that handle mathematical formulation, semantic translation to HDL, and physical-level wave tracking. As illustrated in Figure 2, this framework creates a structured development pipeline transforming circuit topologies into deterministic Verilog implementations.

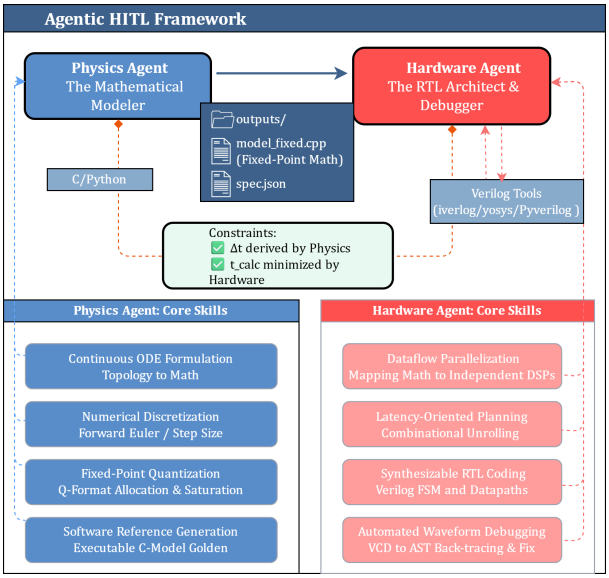


Fig. 1: Overview of the proposed Agentic HiL framework. The Physics Modeling Agent converts circuit topologies into discrete-time numerical models and fixed-point golden references. The Hardware RTL Agent generates synthesizable Verilog implementations and performs waveform-based verification through iterative feedback.

A. Physics-Aware Modeling Agent

Translating a continuous physical topology into hardware requires traversing multiple abstraction layers. The Physics-Aware Modeling Agent supports two complementary numerical formulation routes—State-Space and Associated Discrete Circuit (ADC) with Modified Nodal Analysis (MNA).

State-Space route: The agent first formulates continuous-time differential equations from the circuit topology via KVL/KCL. Given a user-specified or system-driven integration timestep (Δt)—constrained by the target FPGA clock frequency, switching frequency, and circuit dynamics—the agent then applies a numerical discretization method (e.g., Forward Euler or Backward Euler) to obtain the recursive update:

$$x[k+1] = \Phi_{M_s} \cdot x[k] + \Gamma_{M_s} \cdot u[k] \tag{1}$$

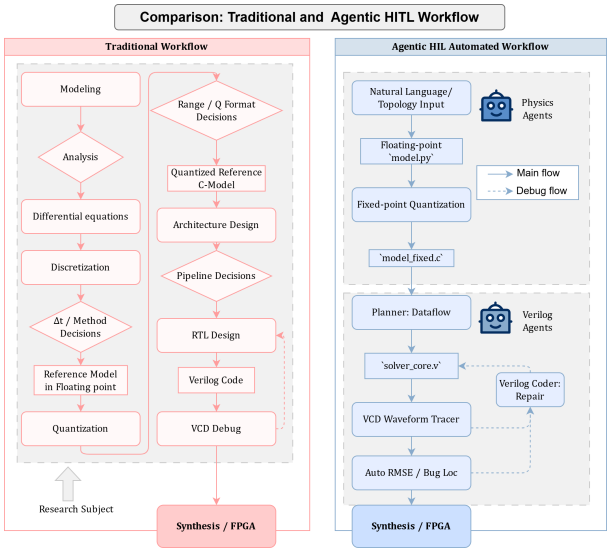


Fig. 2: Comparison of the traditional HiL workflow (left) and the proposed Agentic HiL workflow (right). The manual path requires iterative HDL coding, functional simulation, timing closure, and board-level debugging across multiple engineering domains. The automated path integrates these iterative steps into an automated multi-agent workflow, with the golden reference replacing manual testbench authoring.

The agent supports both explicit (Forward Euler) and implicit (Backward Euler) discretization schemes, allowing the trade-off between computational efficiency and numerical stability to be evaluated. This route is suited for circuits where the number of switching combinations remains tractable (e.g., low switch-count topologies with ideal switches). The state matrices corresponding to the 2^n switching configurations are precomputed offline and selected through a lookup table during online execution, avoiding real-time matrix inversion.

ADC+MNA route: The agent first discretizes each dynamic component into its Norton equivalent. Following the passive sign convention and Backward Euler discretization, an inductor becomes a conductance $G_L = h/L$ in parallel with a current source $J_L = \pm i_L[k]$, and a capacitor becomes a conductance $G_C = C/h$ in parallel with a current source $J_C = -G_C \cdot v_C[k]$. These conductances and history sources are then stamped into a nodal admittance matrix $\mathbf{Y}$. Applying KCL yields the algebraic system:

$$\mathbf{Y} \cdot \mathbf{v} = \mathbf{i} \tag{2}$$

Because $\mathbf{Y}$ remains structurally fixed across switch transitions—the ADC companion formulation preserves a constant system matrix structure by representing switching actions through controlled sources rather than rebuilding the nodal matrix—this route requires less memory [14], [15] and scales naturally to high-dimensional circuits. The agent selects the appropriate route based on circuit complexity, target platform, and real-time constraints, representing two

widely adopted real-time electromagnetic transient simulation approaches.

Regardless of the route chosen, the agent then conducts a static variable range analysis to determine suitable fixed-point Q-format representations and generates a C-based Golden Reference, where quantization feasibility is verified before the hardware translation phase. The agent cross-validates all three numerical routes—SS+FE, SS+BE, and ADC+MNA+BE—against each other; any significant discrepancy (e.g., RMSE exceeding a user-specified threshold) flags potential numerical issues in discretization, matrix formulation, or Q-format selection before RTL generation proceeds.

B. Waveform-Driven Hardware Debugging Agent

Unlike conventional digital verification approaches that primarily evaluate logical correctness, this specialized agent optimizes for numerical equivalence between the software formulation and its hardware implementation. Since the fixed-point representation has been determined during the physics modeling stage, this agent focuses on mitigating structural RTL mapping failures. It acts as an autonomous substitute for the tedious manual phase where human engineers typically comb through Value Change Dump (VCD) traces to identify hardware-specific math bugs, such as bit-misalignment, missing sign-extensions, or unhandled register wrap-around overflows.

When simulating the generated RTL Verilog, the agent cross-verifies its output against the Golden Reference. If severe numerical discrepancies occur (e.g., RMSE of a current waveform spikes abruptly due to improper bit-shifting or DSP multiplier wrap-around), the agent traces backward through the RTL syntax hierarchy and AST representation, pinpoints the precise arithmetic node, and auto-corrects the datapath definitions.

C. Automated Hardware Architecture & Synthesis Loop

A profound domain-specific challenge in HiL simulators is that the objective is *ultra-low hardware latency* (ensuring $t_{calc} \ll \Delta t$), rather than *high algorithmic throughput*. The agent therefore iteratively determines the minimum pipeline depth needed to satisfy timing closure at the target clock frequency, driven by the chosen fabric primitive. For DSP48E1-based designs the native multiply-add pipeline is retained; for LUT-dominated designs the datapath is optimized using shallow combinational structures. If timing closure fails, the automated loop restructures the datapath (e.g., inserting pipeline stages only where needed) instead of blindly extending depth.

III. Experimental Validation

A. Buck Converter (DCM)

The proposed framework was validated on a diode-rectified Buck converter operating in discontinuous conduction: $V_{in} = 24$ V, $L = 100$ μH, $C = 100$ μF, $R_L = 10$ Ω, $f_{sw} = 20$ kHz, $D = 0.5$. The generated solver (State-Space + Backward Euler, 50 ns step) was cross-validated against PLECS and an alternative ADC+MNA implementation.

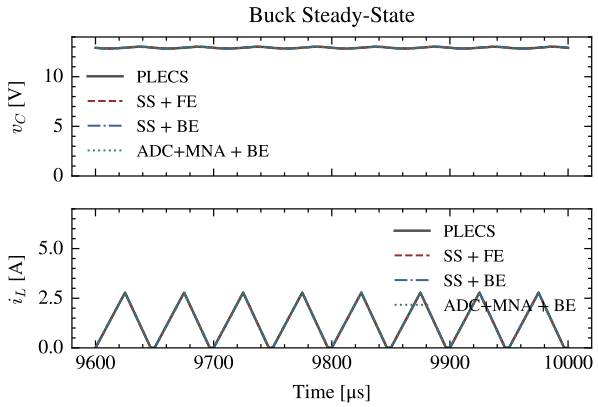


Fig. 3: Buck steady-state: PLECS, State-Space (FE/BE), and ADC+MNA (BE) overlay. All four traces are visually indistinguishable. The inductor current i_L exhibits DCM behavior—zero-current clamping during the diode-blocking interval.

Crucially, the hardware agent demonstrates physics-aware handling of converter operating modes by automatically identifying the Discontinuous Conduction Mode (DCM) and selecting the corresponding piecewise-linear state equations to enforce zero-current boundary conditions. As shown in Figure 3, the steady-state DCM waveform—charge, freewheel, and dead-time phases—is identically reproduced across all solvers.

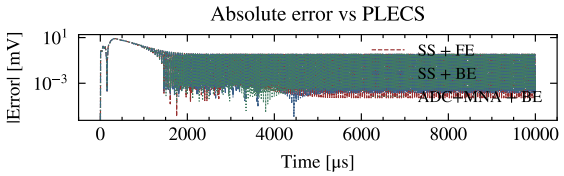


Fig. 4: Absolute error vs. PLECS for the Buck converter. All three Python implementations converge to within 0.19 mV of the commercial reference.

The steady-state error is evaluated over the final 2 ms of the 10 ms simulation window, corresponding to roughly ten RC time constants after startup. This ensures the comparison isolates fixed-point quantization effects rather than transient dynamics. During the startup transient all solvers overlay identically (Figure 5), confirming that numerical errors remain bounded and diminish as the system approaches equilibrium.

All Python solvers agree with PLECS to within 0.19 mV (Figure 4), and the three numerical formulations exhibit negligible deviation from each other at the selected 50 ns timestep (well below 1 μV in steady state, detailed in Table III), with the startup transient identically reproduced across all solvers (Figure 5).

a) Fixed-point quantization.: The Physics Agent sweeps the fractional bit width to select a suitable Q-format balancing accuracy and resource utilization for each topology. For Buck, Q7.17 (24-bit including sign bit, range ± 64 V) is selected

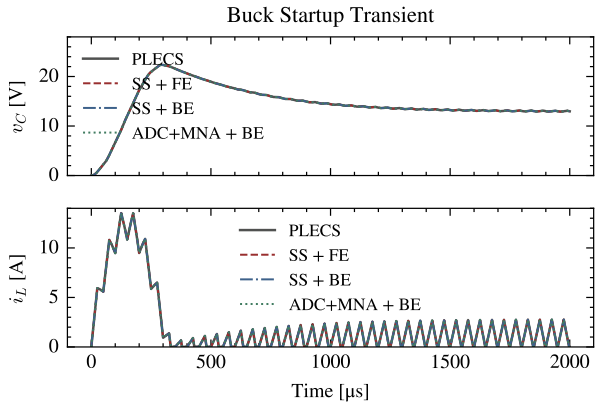


Fig. 5: Buck startup transient (2 ms window). All four solvers (PLECS, SS+FE, SS+BE, ADC+MNA+BE) overlay identically, demonstrating the DCM zero-current clamping behavior from the first switching cycles.

to accommodate startup inrush without saturation, keeping numerical error below 4.87 mV. Wider formats halve the error at the cost of an additional DSP slice, while narrower formats degrade error by an order of magnitude (Table I).

TABLE I: Buck fixed-point quantization sweep.

Q-Fmt	Width	Err [mV]	DSP
Q7.25	32-bit	0.56	2
Q7.21	28-bit	0.56	2
Q7.17	24-bit	4.87	**1**
Q7.13	20-bit	97.2	1

The agent-generated RTL deployed on the Zynq-7020 reproduces these waveforms on hardware, as confirmed by oscilloscope measurements in Figure 6.

B. Boost Converter (CCM)

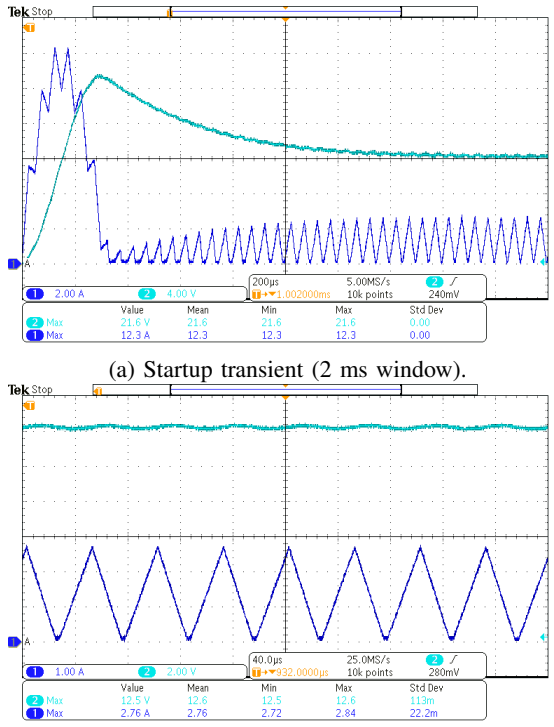
The framework was further evaluated on a Boost converter operating in continuous conduction mode (CCM): $V_{in} = 100$V, $L = 1$ mH, $C = 470$ μF, $R = 20$ Ω, $f_{sw} = 20$ kHz, $D = 0.5$. The underdamped L-C resonance ($\zeta \approx 0.036$, $\tau \approx 18.8$ ms) stresses the solver over long time scales. This topology builds on the FPGA-based real-time simulation methodologies established in the literature [4].

The Boost results are summarized in Table III. All formulations agree with PLECS to within 3.15 mV, and the FPGA implementation reproduces these waveforms on hardware (Figure 9).

For this topology the agent selects Q9.15 (24-bit including sign bit, range ± 256 V) to accommodate the higher output voltage. The fixed-point quantization sweep is shown in Table II.

C. FPGA Implementation

The solver employs a dual-clock architecture: a fast computational clock (180 MHz) drives the solver pipeline and PWM generation, while a 20 MHz clock-enable domain



(a) Startup transient (2 ms window).

(b) Steady-state (400 μs window).

Fig. 6: Oscilloscope measurements of the agent-generated RTL running on Zynq-7020 at 20 MHz solver rate (50 ns step). CH1 (yellow): output voltage v_C (5 V/div for startup, 2 V/div for steady-state); CH2 (blue): inductor current i_L (2 A/div for startup, 1 A/div for steady-state).

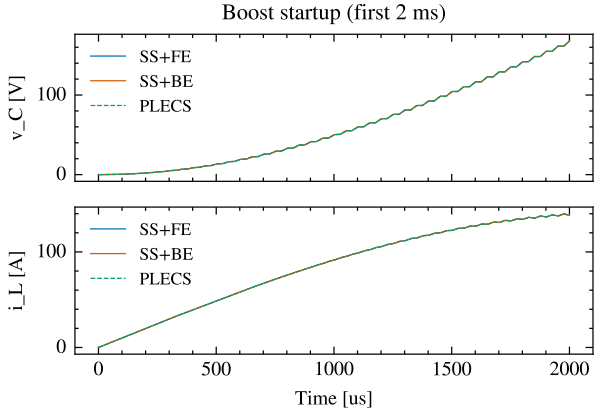


Fig. 7: Boost startup transient. All methods are indistinguishable, tracking the inrush current buildup and resonant voltage rise identically.

TABLE II: Boost fixed-point quantization sweep.

Q-Fmt	Width	Err [mV]	DSP
Q9.23	32-bit	3.02	2
Q9.19	28-bit	3.39	2
Q9.15	24-bit	7.45	**1**
Q9.13	22-bit	40.8	1

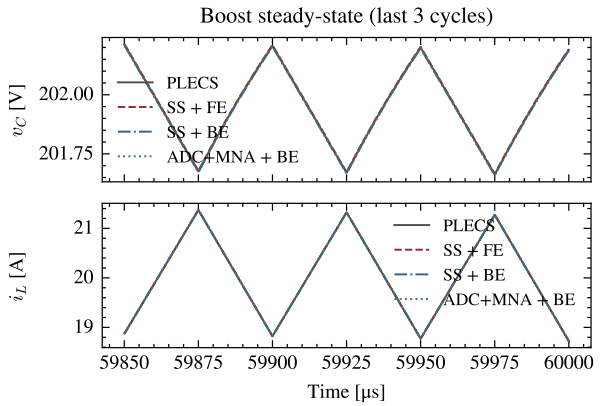
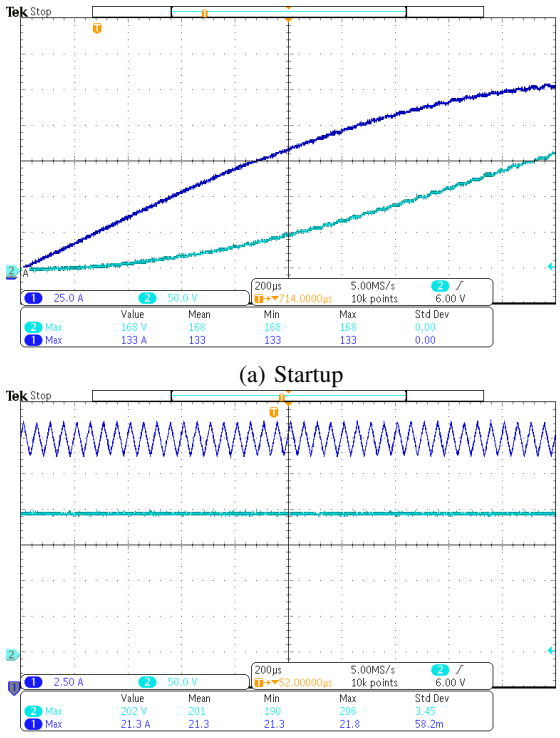


Fig. 8: Boost steady-state (last three switching cycles at $t = 60$ ms). The CCM triangular current ripple ($\Delta i_L \approx 2.65$ A p-p) is captured identically by all solvers.



(a) Startup

(b) Steady-state

Fig. 9: Oscilloscope measurements of the agent-generated Boost RTL on Zynq-7020 at 20 MHz solver rate. CH1 (yellow): output voltage v_C; CH2 (blue): inductor current i_L. Waveforms match the simulated startup transient and steady-state ripple.

TABLE III: Cross-Validation: All Methods vs. PLECS

Method	**Buck**		**Boost**	
	v_C [V]	MAE [mV]	v_C [V]	Err [mV]
PLECS (ref.)	12.92612	—	201.9418	—
SS + FE	12.92615	0.19	201.9448	2.98
SS + BE	12.92609	0.19	201.9388	2.98
ADC+MNA + BE	12.92609	0.19	201.9386	3.15

derived from the 180 MHz solver clock governs the I/O interface at the 50 ns HiL step rate. Both domains share the same 180 MHz clock, eliminating the need for clock-domain-crossing synchronizers. The target device is a Xilinx Zynq-7020 (xc7z020clg400-2) featuring an Artix-7 FPGA fabric with 85,000 logic cells, 53,200 LUTs, 106,400 flip-flops, 4.9 Mbit block RAM (140 $\times$ 36 Kb), and 220 DSP48E1 slices. On this device the agent generates two solver variants reflecting different primitive choices. The Buck solver uses 3 DSP48E1 slices with a 4-stage pipeline that absorbs the DSP's native multiply-add latency (22.2 ns total), closing timing at 180 MHz with $+0.201$ ns slack. The Boost solver uses LUT-based multipliers (0 DSP slices) and operates with shallow combinational logic (5.56 ns). In both cases the computational latency stays well below the 50 ns HiL step, leaving > 44 ns for I/O. The pipeline timing is visualized in Figure 10: the 4-stage latency of 22.22 ns is clearly visible within the 50 ns HiL step.

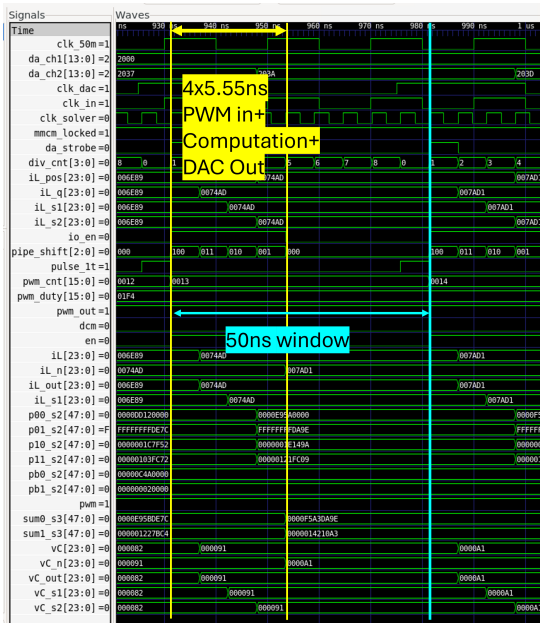


Fig. 10: GTKWave simulation of the Buck solver pipeline (200 ns window). From top: `solver_clk` (180 MHz), `io_en` (enable at 20 MHz rate), pipeline stages S1–S4 showing the 4-cycle latency (22.22 ns), and `da_strobe` (DAC update pulse). The pipeline latency occupies $< 45\%$ of the 50 ns HiL step, leaving > 27 ns for I/O operations.

Resource utilization is summarized in Table IV.

Each agent-generated solver comprises approximately 200 lines of synthesizable Verilog for the numerical solver core and 200 lines of board-level integration (clocking, clock domain crossing (CDC), DAC), totaling under 500 lines of RTL per topology. The equivalent manually-coded implementation typically requires several days of expert effort, whereas the multi-agent framework significantly reduces the manual effort required for RTL development. Similarly, MATLAB HDL Coder provides mature model-based HDL gen-

TABLE IV: Agent-Generated RTL Resource Utilization on XC7Z020

Topology	Resource	Used	Total	%
Buck	LUTs	156 (core: 128)	53,200	0.29%
	FFs	264 (core: 187)	106,400	0.25%
	DSP48E1	3	220	1.4%
Boost	LUTs	61	53,200	0.11%
	FFs	57	106,400	0.05%
	DSP48E1	0	220	0%
Solver clock: 180 MHz (5.56 ns, slack +0.01 ns) I/O rate: 20 MHz (50 ns = HiL step)				

eration and fixed-point support, but still relies on manually constructed Simulink/Simscape models and engineer-driven workflow configuration. The proposed method reduces the dependency on these manual modeling steps.

IV. Conclusion and Future Work

This paper introduces Agentic HiL, a multi-agent LLM paradigm that demonstrates the feasibility of bridging continuous-time power electronic models and ultra-low latency FPGA hardware. By addressing domain-specific challenges—such as numerical quantization, DCM physical mapping, and sub-microsecond latency flattening—the system mitigates the complexities of VHDL/Verilog manual coding. The proposed framework, including its full repository of LLM skill-sets and automated synthesis scripts, will be considered for release after publication.

a) Limitations.: The current framework targets engineers with HiL development experience rather than serving as a turnkey solution for novices. LLM-generated RTL requires human review before deployment, though the review effort is substantially lower than manual Verilog authoring from scratch. Validation is currently limited to two-state converter topologies (buck, boost); higher-order systems such as DAB or resonant converters may require human guidance for pipeline depth selection and numerical stiffness handling. These limitations are not fundamental—they reflect the current maturity of the approach and are addressable through planned extensions including SPICE netlist integration, MATLAB toolchain coupling via MCP server protocols, and composable skill pipelines that cascade LTSpice parsing, HiL generation, and automated test execution.

The full paper will expand this validation to include high-complexity nonlinear topologies, such as a Dual Active Bridge (DAB) and a Resonant LLC Converter. Moreover, the agent-generated solver consumes minimal fabric resources (129 LUTs, 3 DSP48E1s), leaving ample room for RCP controllers such as UltraZohm [8] on the same device. This enables unified RCP+HiL preliminary validation on a single commodity FPGA without dedicated real-time simulator hardware.

Currently the MNA formulation is derived by the LLM from natural-language topology descriptions, which can introduce subtle errors in the conductance matrix or companion-model mapping. Future work will adopt a structured netlist-based approach—parsing standard SPICE netlists to automatically generate the MNA stamps—eliminating this ambiguity and improving cross-route consistency.

References

[1] X. Yuan, "Opportunities, challenges, and potential solutions in the application of fast-switching sic power devices and converters," *IEEE Transactions on Power Electronics*, vol. 36, no. 4, pp. 3925–3945, 2021.

[2] G. Lauss and K. Strunz, "Accurate and stable hardware-in-the-loop (hil) real-time simulation of integrated power electronics and power systems," *IEEE Transactions on Power Electronics*, vol. 36, no. 9, pp. 10 920–10 932, 2021.

[3] A. Myaing and V. Dinavahi, "Fpga-based real-time emulation of power electronic systems with detailed representation of device characteristics," *IEEE Transactions on Industrial Electronics*, vol. 58, no. 1, pp. 358–368, 2011.

[4] H. Bai, C. Liu, E. Breaz, K. Al-Haddad, and F. Gao, "A review on device-level real-time simulation of power electronic converters," *IEEE Industrial Electronics Magazine*, vol. 15, no. 1, pp. 12–27, 2021.

[5] H. Bai, C. Liu, D. Majstorovic, and F. Gao, *Real-Time Simulation Technology for Modern Power Electronics*. Academic Press (Elsevier), 2023.

[6] T. Cheng and V. Dinavahi, "A device-level transient modeling approach for fpga-based real-time simulation of power converters," *IEEE Transactions on Power Electronics*, vol. 35, no. 8, pp. 8078–8091, 2020.

[7] ——, "An fpga-based igbt behavioral model with high transient resolution for real-time simulation of power electronic circuits," *IEEE Transactions on Industrial Electronics*, vol. 66, no. 10, pp. 7810–7820, 2019.

[8] E. Liegmann, T. Schindler, P. Karamanakos, A. Dietz, and R. Kennel, "Ultrazohm—an open-source rapid control prototyping platform for power electronic systems," in *International Aegean Conference on Electrical Machines and Power Electronics (ACEMP) & International Conference on Optimization of Electrical and Electronic Equipment (OPTIM)*, 2021, pp. 445–450.

[9] B. Stellato, T. Geyer, and P. J. Goulart, "High-speed finite control set model predictive control for power electronics," *IEEE Transactions on Power Electronics*, vol. 32, no. 5, pp. 4007–4020, 2016.

[10] Q. Sui, B. Du, Y. Zuo, and W. Martinez, "Exploring the potential of fpga in high-frequency switching dc-dc boost converters using model predictive control," in *IEEE Applied Power Electronics Conference and Exposition (APEC)*, 2025, pp. 2752–2756.

[11] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu *et al.*, "Autogen: Enabling next-gen llm applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[13] C.-T. Ho, H. Ren, and B. Khailany, "Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (ast)-based waveform tracing tool," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 1, 2025, pp. 300–307.

[14] M. Milton, A. Benigni *et al.*, "Latency insertion method based real-time simulation of power electronic systems," *IEEE Transactions on Power Electronics*, vol. 33, no. 12, pp. 10 752–10 764, 2018.

[15] Y. Chen and V. Dinavahi, "A network analysis modeling method of power electronic converters for hardware-in-the-loop applications," *IEEE Transactions on Transportation Electrification*, vol. 5, no. 3, pp. 709–719, 2019.